RahiScan

Automated Rickshaw Fare & Pace Telemetry

A Data Science Lifecycle Case-Study Proposal

Introduction to Data Science
Group Assignment
Submitted 21 October 2026
`github.com/ZEKT-VII/RahiScan-DS-Proposal`

Group Contributions Form

This form records how our three-member team divided the work behind this proposal. Each member led specific stages of the Data Science Lifecycle and reviewed the stages led by the others, so the whole report reflects a shared design.

Member	Name	Roll Number	Contribution
Member 1	**Muhammad Saad Hussain**	SU92-BSCSM-F25-260	Wrote the case-study story, framed the problem, designed the scanning workflow and data-collection schema, and drew the system architecture. Led the **Define** and **Collect** stages.
Member 2	**Muhammad Khubaib**	SU92-BSCSM-F25-300	Built the mock Lahore dataset and Python pipeline, wrote the OCR and walking-tail cleaning rules, derived the Pace Index and Fare Ratio, and set up the gradient-boosting models. Led **Clean**, **Explore** and **Model**.
Member 3	**Huzaifa**	SU92-BSCSM-F25-408	Designed the holdout evaluation and success thresholds, produced the charts, planned the in-app deployment and privacy safeguards, maintained the GitHub repository and checked formatting. Led **Evaluate** and **Deploy**.

Lifecycle Ownership Matrix

	Define	Collect	Clean	Explore	Model	Evaluate	Deploy
M. Saad Hussain	**Lead**	**Lead**	Review	Support	Review	Review	Support
M. Khubaib	Support	Support	**Lead**	**Lead**	**Lead**	Support	Review
Huzaifa	Review	Review	Support	Support	Support	**Lead**	**Lead**

Deliverables Submitted

This report (PDF)	Case-study story, lifecycle extraction, mock results and the proposed plan.
Mock dataset	1,224 ride submissions, 1,200 scans and 364,488 GPS pings from 40 simulated drivers (`data/`).
Reproducible code	Data generator, cleaning and modelling pipeline, and figure scripts in Python (`src/`).
Repository	`https://github.com/ZEKT-VII/RahiScan-DS-Proposal`

M. Saad Hussain M. Khubaib Huzaifa

1. The Case Study: A Morning on Raiwind Road

It is 7:40 on a Monday and the three of us are standing outside a bus stop in Johar Town with a 8:30 lab ahead. The first auto-rickshaw driver asks for 600 rupees to reach campus. We know the ride should cost about 450, so we argue, settle at 520 and climb in. The driver then crawls along the left lane at walking pace, stops for tea at a roadside dhaba "for one minute", and we walk into the lab twelve minutes late. The next morning a different driver takes the same route for 440 rupees and gets us there in twenty minutes, even though the traffic was worse.

None of us could have told these two drivers apart from the curb. Lahore's rickshaws have no meters, so the price is whatever the driver names, and a passenger has no way to know in advance whether a driver is slow by habit or only stuck in traffic. Every student who commutes this way faces the same information gap, and every bad ride leaves behind data that nobody records.

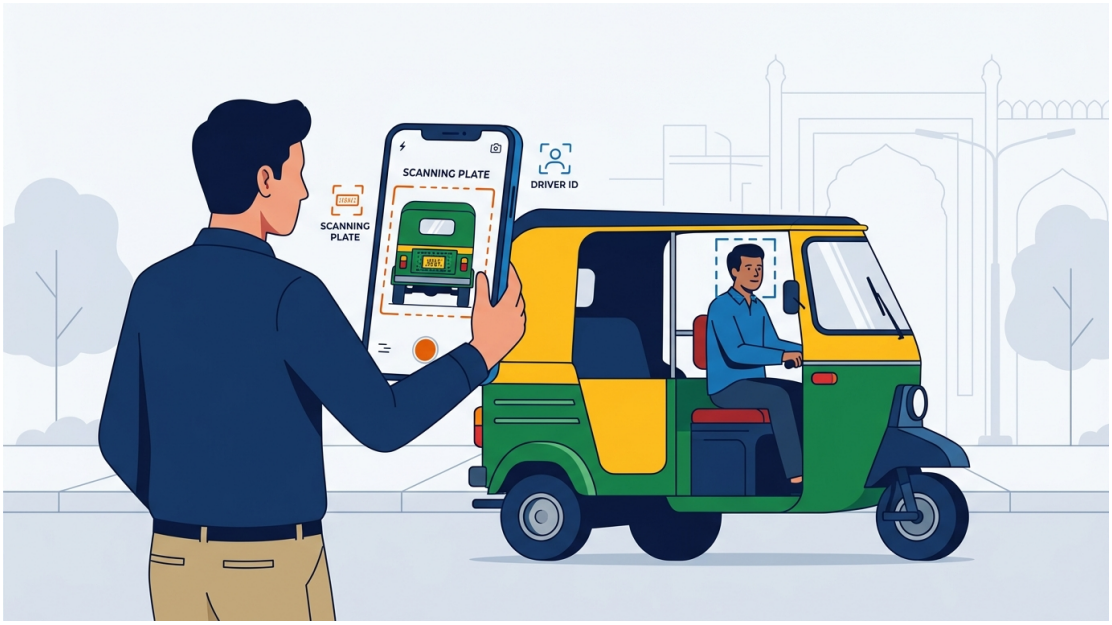


Figure 1: The proposed RahiScan interaction. Before boarding, the passenger photographs the rickshaw. The app reads the number plate (orange box) and matches the driver's face (blue box), then returns that driver's history of pace and pricing.

Our proposal. We propose to build **RahiScan**, a crowd-sourced mobile app that collects this missing data. A passenger photographs the rickshaw before boarding. The app identifies the vehicle from its Punjab excise number plate and the driver from their face, because many rickshaws are rented by two drivers on different shifts. The passenger types in the quoted fare and starts the ride. From then until arrival the app logs a GPS position every five seconds. After many rides, each driver will have a measured record of how fast they drive compared with the traffic around them, and how their prices compare with a fair benchmark. The next passenger who scans that rickshaw sees the record before agreeing to get in.

The rest of this report takes the story apart stage by stage. For each stage of the Data Science Lifecycle (Lecture 02) we name the problem the story raises, describe how we will handle it, and show what a simulated four-week pilot on *mock data* suggests the result will look like. Every number in this report comes from that simulation, and the dataset and code are in our GitHub repository.

2. Extracting the Data Science Lifecycle from the Story

The story contains seven separate data science problems, one for each lifecycle stage. Figure 2 shows the stages and the guiding question each one answers, and Table 1 traces every problem back to the moment in the story that raised it.

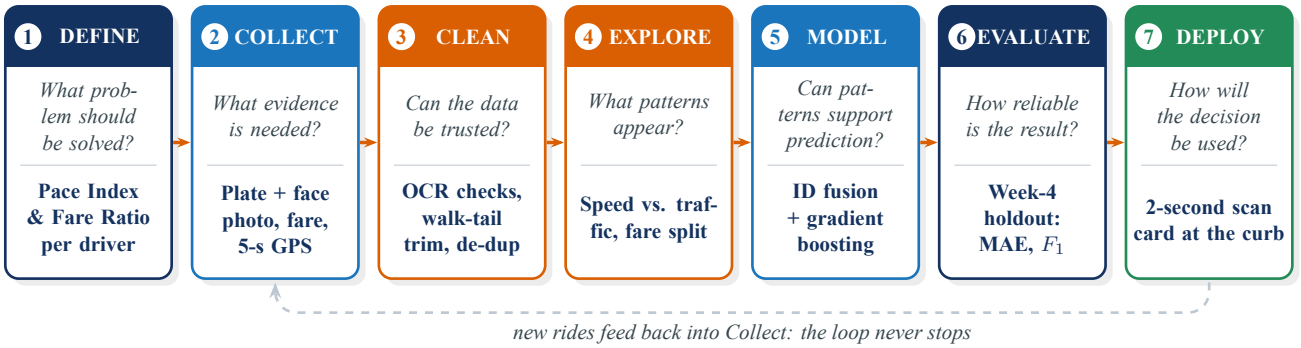


Figure 2: The seven-stage Data Science Lifecycle applied to RahiScan. Each card pairs the lecture's guiding question with our planned answer.

Table 1: Data science problems extracted from the case-study story.

Stage	*Moment in the story*	Data science problem it raises
Define	*"We could not tell the two drivers apart."*	"Good" and "bad" are vague. We need measurable targets: a speed score that accounts for traffic and a price score against a fair benchmark.
Collect	*"Every bad ride leaves behind data nobody records."*	What evidence proves who the driver is, what was charged and how the ride went? We need photos, the fare quote, GPS traces and reference fares.
Clean	*"We walked into the lab twelve minutes late."*	Passengers who forget to tap *End Ride* log their walk to class as ride time. Blurry plates, duplicate uploads and fare typos also corrupt the records.
Explore	*"...even though the traffic was worse."*	Raw speed blames drivers for traffic jams. Speed has to be compared with what other rickshaws managed on the same road at the same hour.
Model	*"A different driver took the same route."*	Identify the right driver, including on shared rickshaws, and predict a fair fare and a pace class from their history.
Evaluate	*"A driver stuck in traffic is not a slow driver."*	How often would the app wrongly label an honest driver? We need error rates on rides the model has never seen.
Deploy	*"We had to decide at the curb, in seconds."*	The answer must reach the passenger as a clear card within two seconds, while protecting driver privacy and allowing appeals.

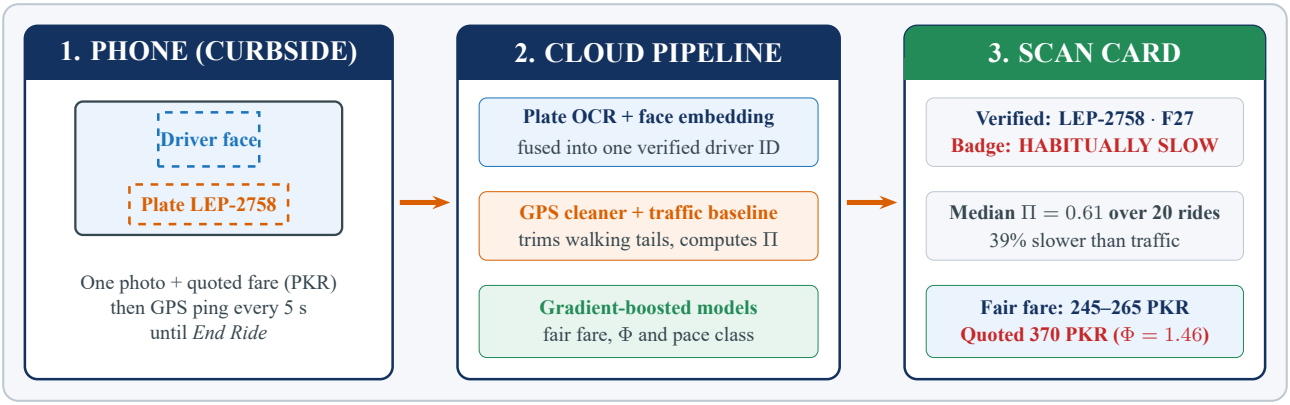


Figure 3: Proposed system architecture. The scan card shows a real driver profile from our mock pilot (driver D27).

3. Define: Turning Frustration into Measurable Targets

Stage 1 · DEFINE *What problem should be solved?*

"This driver is slow" and "this driver overcharges" are opinions. To make them testable we define two ratios for every ride i. A value of 1 means typical, so the ratios can be compared across routes and hours.

Driving Pace Index

$$\Pi_i = \frac{v_{\text{obs},i}}{v_{\text{base},i}} = \frac{d_i/t_i}{\text{median}(v_{\text{obs}} \text{ on corridor, hour})}$$

A driver at $\Pi = 0.6$ moves 40% slower than other rickshaws managed on the same road at the same time.

Fare Fairness Ratio

$$\Phi_i = \frac{\text{Fare}_{\text{quoted},i}}{\widehat{\text{Fare}}_{\text{fair},i}}$$

$\widehat{\text{Fare}}_{\text{fair}}$ is predicted from distance, corridor and congestion, using ride-hailing quotes for the same trip as the benchmark.

The app turns these ratios into badges. We fixed the thresholds before looking at any data so they cannot be tuned to flatter the results (Table 2). A badge describes a driver's median over at least five rides, never a single ride.

Table 2: Decision rules defined before data collection.

Badge	Rule (driver median)	What the passenger is told
Habitually Slow	$\Pi < 0.75$	Expect a ride at least 25% longer than traffic requires.
Normal Pace	$0.75 \le \Pi \le 1.15$	Moves with traffic.
Fast	$\Pi > 1.15$	Noticeably quicker than traffic.
Overpriced	$\Phi > 1.25$	Usually quotes more than 25% above the fair fare. Negotiate.

4. Collect: The Evidence We Need

Stage 2 · COLLECT *What evidence is needed?*

No public dataset links a Lahore rickshaw driver to their speed and prices, so RahiScan will collect its own. We plan a four-week pilot with about 85 student volunteers on eight commuter corridors around

campus. Table 3 lists the five data streams. We designed each one around a question the later stages must answer.

Table 3: Planned data streams and the volume produced by our simulated pilot.

Stream	Fields captured	Why we need it	Mock rows
Curb scan	Plate OCR text and confidence, face-embedding match and similarity, low-light flag	Who is driving?	1,200
Ride record	User, corridor, start and end time, quoted fare (PKR), *End Ride* tapped?	What was charged?	1,224
GPS pings	Latitude, longitude, timestamp every 5 s, accelerometer step variance	How was the ride driven?	364,488
Fare benchmark	inDrive, Careem or Bykea quote for the same origin and destination	What is a fair fare?	1,200
Driver gallery	Plate to driver(s), face ID, shift (34 vehicles, 40 drivers)	Who shares a rickshaw?	40

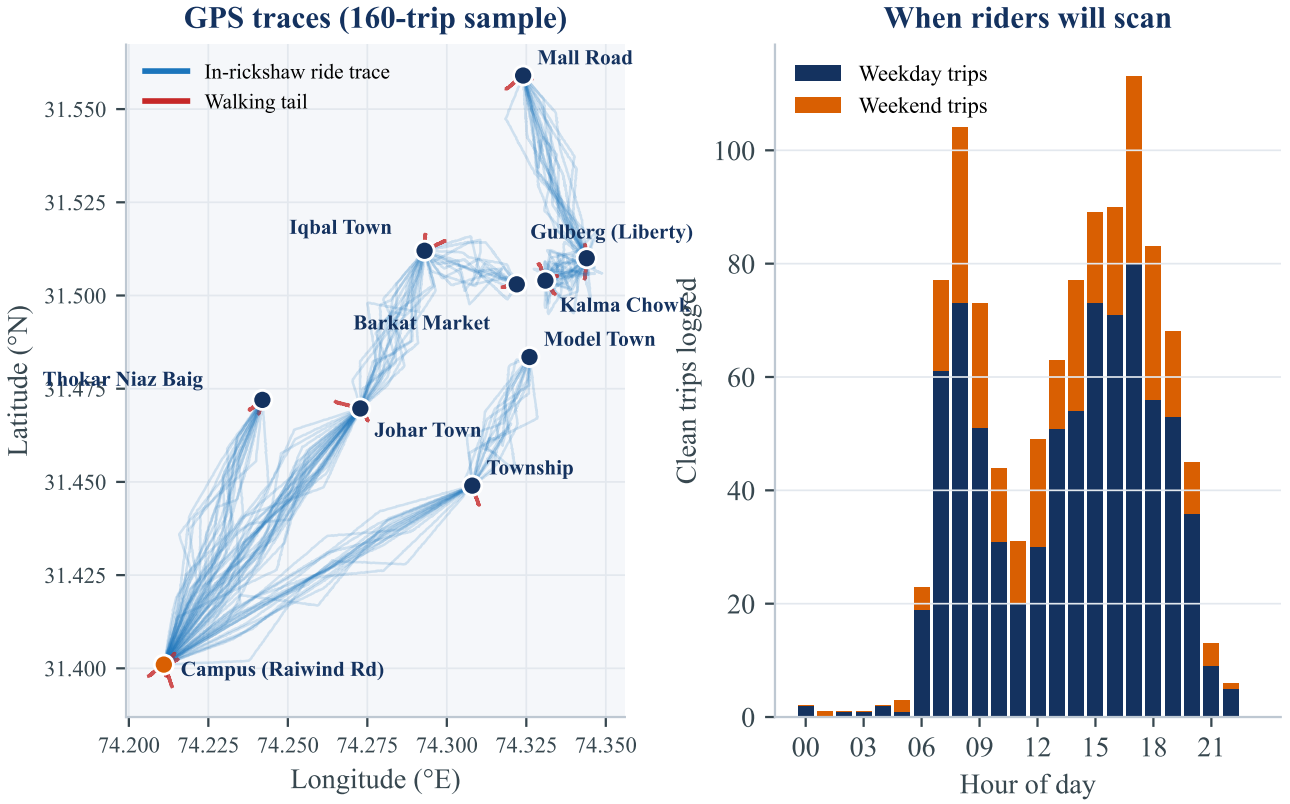


Figure 4: Left: simulated GPS traces on the eight pilot corridors, with walking tails from forgotten *End Ride* taps in red. Right: trips cluster in the morning and evening rush, which is exactly when traffic and driver behaviour are hardest to separate.

Consent and privacy. Face data is sensitive. Drivers will be told about the pilot and may opt out. The app will store only a numeric face embedding, never the photo itself, and pings recorded before boarding or after the ride ends will be deleted once cleaning is complete.

5. Clean: Can the Data Be Trusted?

Stage 3 · CLEAN *Can the data be trusted?*

Crowd-sourced data from phones on a busy street will be messy, so we put realistic faults into the mock data on purpose and wrote one rule for each. **Duplicate uploads** caused by weak mobile signal are dropped when the same user submits the same corridor within ten seconds. **Fare errors**, either blank or with an extra zero typed in (5,200 instead of 520), are removed when the quote is more than four times or less than 0.4 times the benchmark. **Identity errors** come from dented or glared plates at night. We accept a plate read only when OCR confidence is at least 0.85. Otherwise the face match must reach a similarity of 0.70, and when two drivers share a plate the face decides between them. Rides with less than 75% **GPS coverage**, usually from underpasses, are excluded. Figure 5 shows how many records each rule removes, and shows that adding the face check raises identification precision from 85.7% to 99.8%.

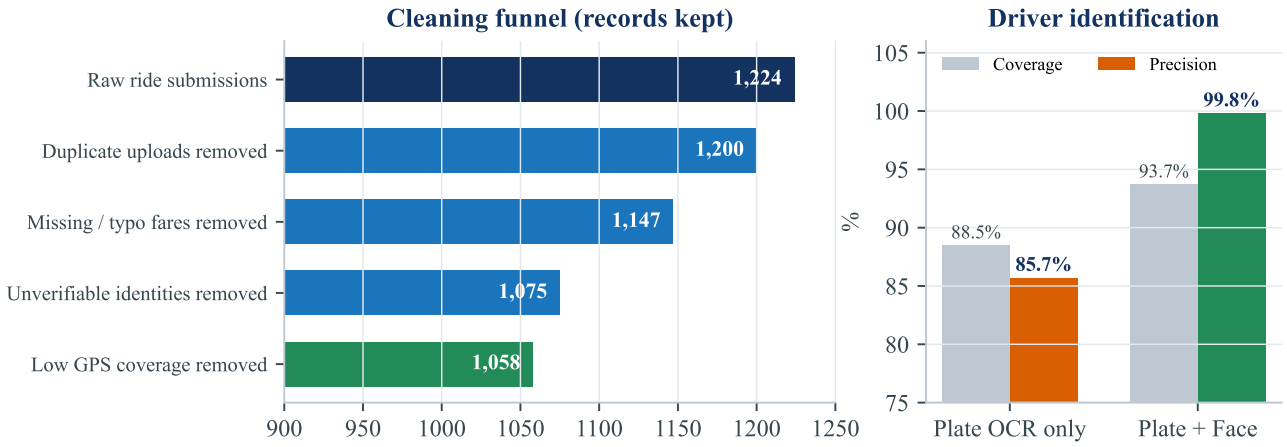


Figure 5: Left: 1,224 raw submissions shrink to 1,058 trusted rides. Right: fusing face and plate both identifies more rides (coverage) and makes fewer mistakes (precision) than reading the plate alone.

The most damaging fault is the **walking tail** from our own story. A student who jumps out and forgets to tap *End Ride* keeps recording a slow walk into the building, which pulls the driver's average speed down. Our filter works back from the end of the trace and cuts every ping where speed stays under 6 km/h *and* the accelerometer shows walking steps for at least a minute. On the mock data it caught 88% of the 130 tails with no false cuts, and it restored a median of 3.1 km/h to each affected driver's speed (Figure 6). Most of the tails it missed had very weak step signals, which matches a phone carried in a bag.

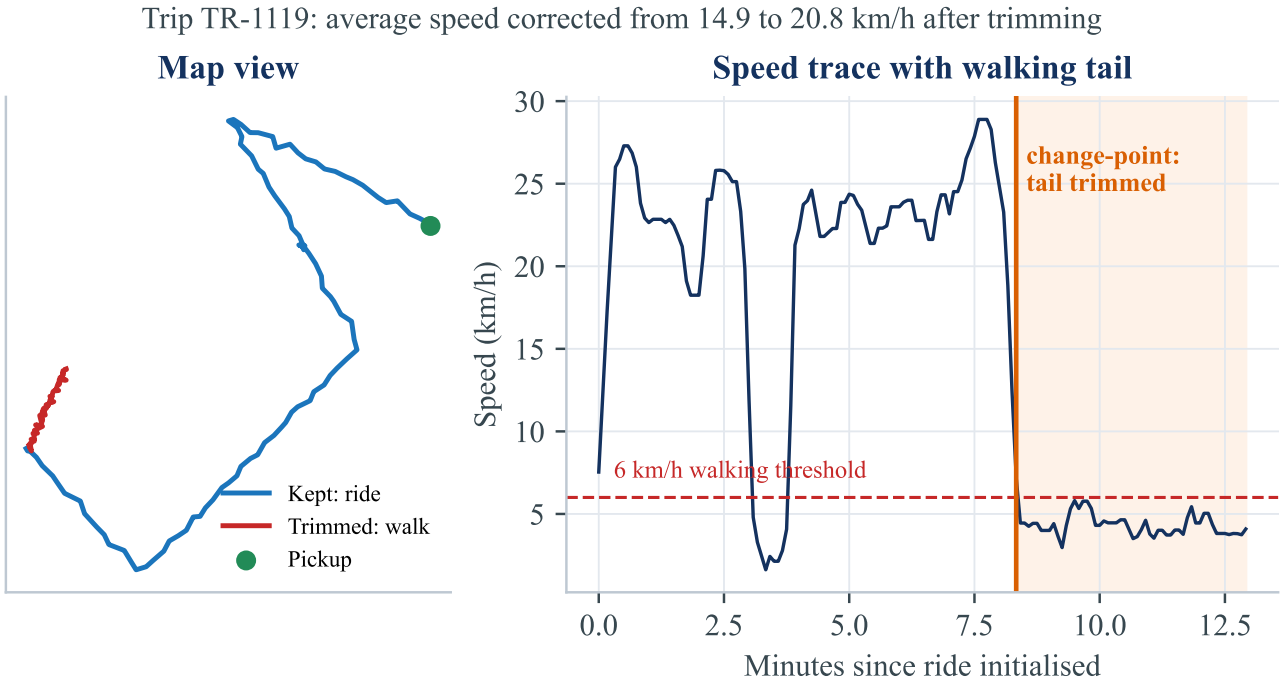


Figure 6: Change-point trimming on one simulated ride. Without the trim, an honest driver would have looked 28% slower than they were.

6. Explore: What Patterns Appear?

Stage 4 · EXPLORE *What patterns appear?*

The first thing to look at is traffic. Figure 7 shows the crowd-sourced baseline speed v_{base}, the median over all drivers, for each corridor and two-hour window. On Mall Road the baseline drops from 22 km/h at night to 5 km/h at 4 pm. A driver doing 7 km/h at that hour is keeping up with traffic. The same speed at 10 pm would be very slow. Raw speed alone cannot tell these cases apart, which is why Π is defined against the baseline.

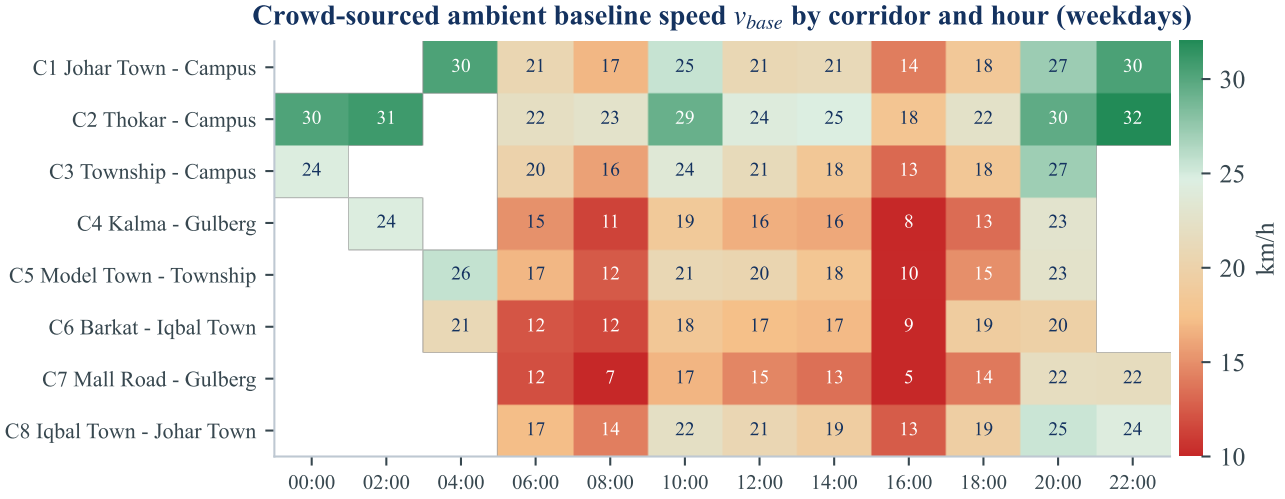


Figure 7: Median ambient speed by corridor and hour on weekdays (mock pilot). Red cells are rush-hour bottlenecks where raw speed would unfairly penalise drivers.

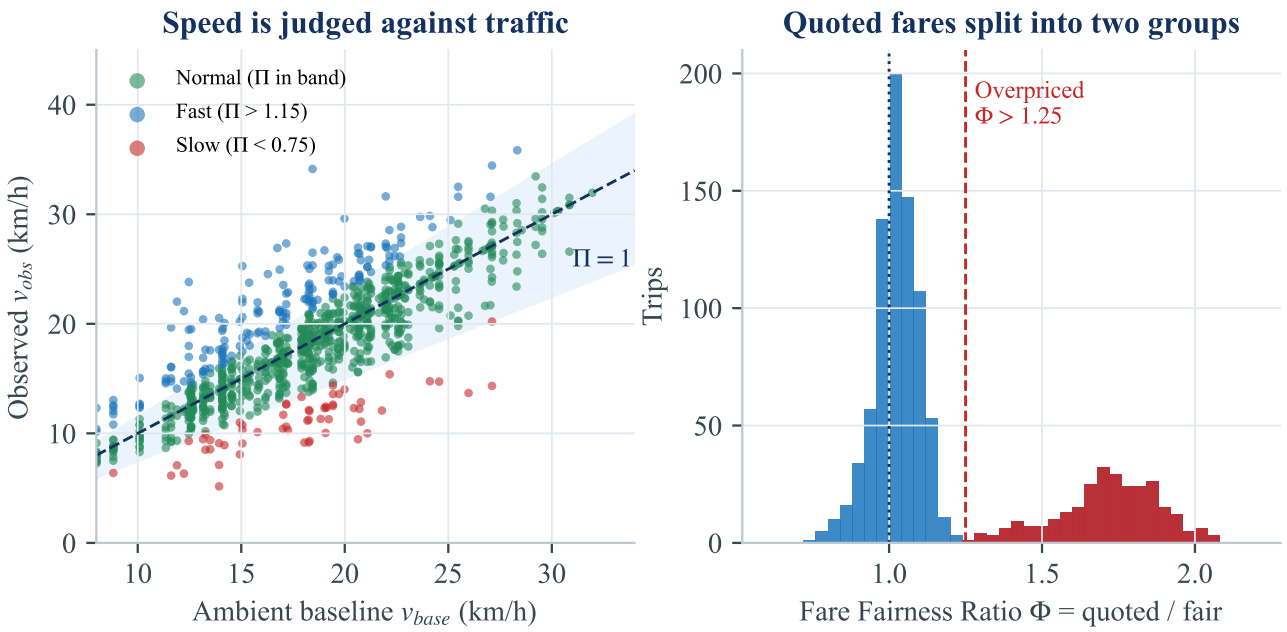


Figure 8: Left: each dot is one ride. Rides inside the blue band move with traffic regardless of absolute speed. Right: the Fare Ratio splits into a fair cluster near 1.0 and an overpriced cluster near 1.7.

Table 4 shows six cleaned rides. Ride TR-0827 moved at only 7.3 km/h, yet $\Pi = 0.91$ because Kalma Chowk was gridlocked at 5 pm, so that driver is not penalised. Ride TR-0597 was slower than traffic *and* quoted 46% above the fair fare. TR-0266 shows a driver who keeps pace but asked for twice the fair fare, and the two scores capture these as separate problems.

Table 4: Sample of the cleaned, feature-engineered mock dataset (`data/processed/trips_clean.csv`).

Ride	Plate # Face	Corridor	Hour	km	v_{obs}	v_{base}	Π	Quoted / Fair	Φ
TR-0466	LEP-4334#F04	Thokar – Campus	14	8.5	25.6	24.6	1.04	460 / 458	**1.00**
TR-0824	LER-5034#F17	Johar Town – Campus	11	9.9	29.1	25.5	1.14	500 / 519	**0.96**
TR-0827	LEP-4877#F22	Kalma – Gulberg	17	2.1	7.3	8.0	**0.91**	180 / 169	**1.07**
TR-0597	LEP-2758#F27	Barkat – Iqbal Town	8	3.9	6.1	11.6	**0.53**	370 / 254	**1.46**
TR-0266	LES-8870#F32	Township – Campus	7	10.8	17.4	18.2	0.96	1070 / 528	**2.03**
TR-0252	LEB-5028#F25	Johar Town – Campus	17	10.1	17.7	14.1	**1.26**	510 / 557	**0.92**

7. Model: From Patterns to Predictions

Stage 5 · MODEL *Can patterns support prediction?*

The modelling plan has two tiers (Figure 9). **Tier 1, identity:** a lightweight MobileNetV3 face encoder and a plate OCR model produce two candidate IDs, and the fusion rule from the Clean stage combines them into one verified driver. This tier matters because in our mock gallery six rickshaws are shared between a day driver and an evening driver. Plate LES-8113, for example, belongs to a fair day driver ($\Phi = 1.05$) and an overpriced evening driver ($\Phi = 1.77$), and a plate-only system would average the two into one misleading score. **Tier 2, behaviour:** a gradient-boosted regressor predicts the fair fare from distance, corridor, hour, day type and v_{base}. A gradient-boosted classifier predicts each ride's pace class, and the driver's badge is the median over their history.



Figure 9: Data journey of a single scan: phone photo, cloud identity models, driver profile database, GPS ride trace, and the scorecard returned to the passenger.

Figure 10 plots every simulated driver's profile after four weeks. Most drivers sit in the fair, normal-pace corner. Eleven form an overpriced cluster, three are habitually slow, and two (`LEP-2758` and `LES-8176`) are both slow and overpriced. That pair is the kind of driver from our first morning.

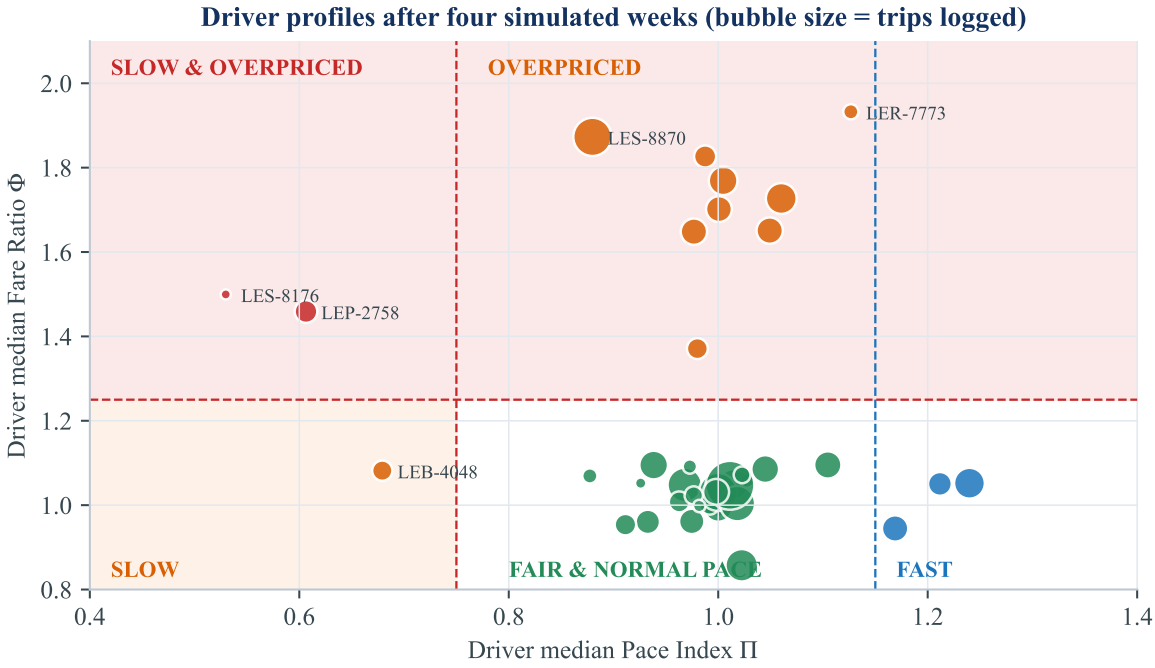


Figure 10: Driver-level profiles in the mock pilot. The two axes are independent: pricing and pace are separate behaviours and need separate badges.

8. Evaluate: How Reliable Is the Result?

Stage 6 · EVALUATE *How reliable is the result?*

We will train on weeks 1 to 3 and test on week 4 only, so every reported score comes from rides the models have never seen, in time order as they would occur in real use. Each module also has a simpler baseline to beat. Table 5 compares the targets we set in advance with the mock-pilot results.

Table 5: Evaluation targets versus simulated results on the week-4 holdout (280 rides, 40 drivers).

Module	Metric (baseline)	Target	Mock result	Status
Driver identity	Precision of verified IDs (plate only: 85.7%)	$\geq 93\%$	**99.8%**	**Met**
Walking-tail filter	Recall of forgotten *End Ride* taps	≥ 0.85	**0.88**	**Met**
Fair-fare regressor	Mean absolute error (linear: 12.1, flat rate: 117.6 PKR)	< 15 PKR	**10.9 PKR**	**Met**
Pace, single ride	Macro F_1 (raw speed only: 0.45)	≥ 0.88	**0.67**	**Not met**
Pace, driver badge	Accuracy over $\geq$ 5 rides (raw speed only: 87.5%)	$\geq 90\%$	**100%**	**Met**
Fare badge	Accuracy of the Overpriced flag	$\geq 90\%$	**100%**	**Met**

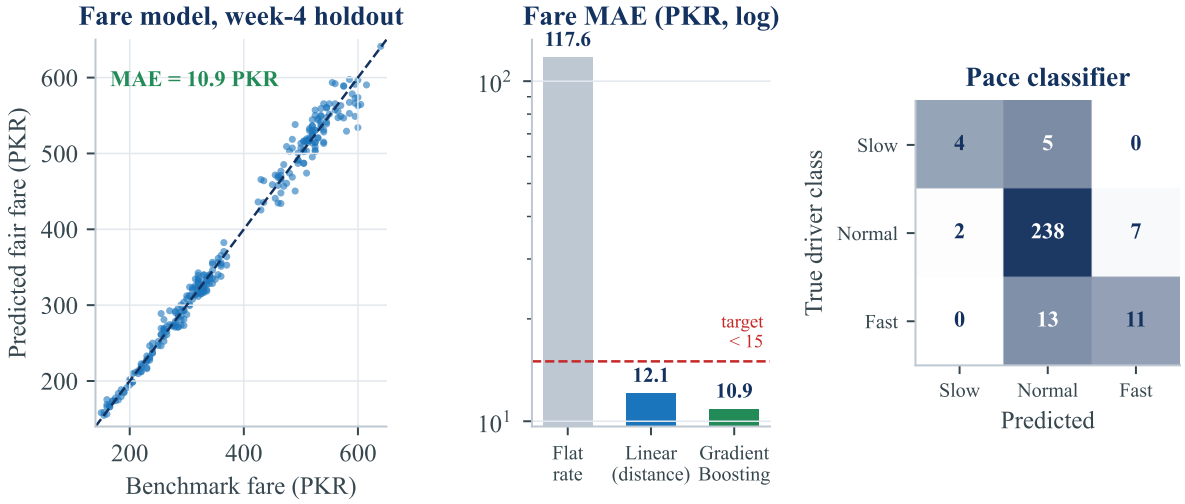


Figure 11: Holdout evaluation. Left and centre: the fare model beats both baselines and stays under the 15 PKR target. Right: single-ride pace predictions mostly confuse Fast and Slow drivers with Normal ones.

The single-ride pace classifier is the one result that misses its target, and the miss changes how we will deploy. One ride is noisy: a slow driver can catch every green light, and a fast driver can sit behind a broken-down bus. Over several rides the median settles, and driver-level badge accuracy reaches 100%, against 87.5% when speed is not compared with traffic (Figure 12). The app will therefore never label a driver from one ride. It will show *Not enough data* until five rides have been logged.

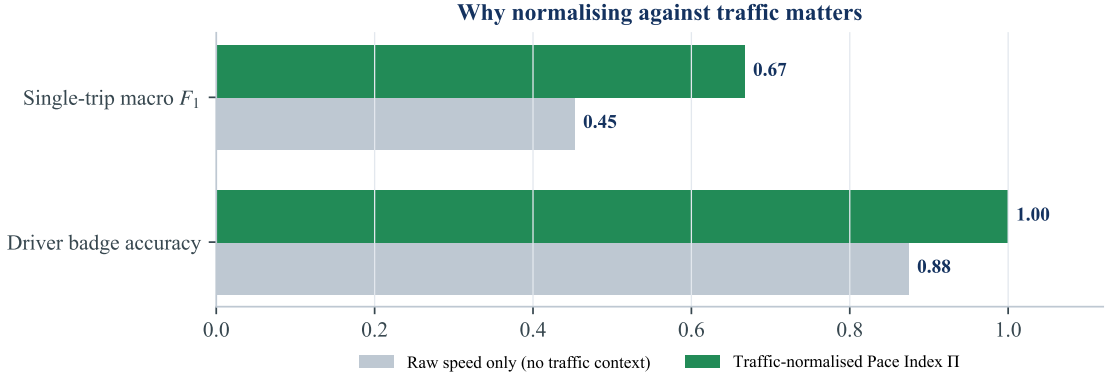


Figure 12: Normalising speed against the crowd-sourced traffic baseline improves results for both single rides and driver badges.

9. Deploy: How Will the Decision Be Used?

Stage 7 · DEPLOY *How will the decision be used?*

The evaluated models will run behind a lightweight cloud API. The passenger's experience has three screens (Figure 13): scan the rickshaw, ride with live tracking, and read a card showing the driver's verified identity, pace badge, and fair-fare range next to the quote. Our target is to return the card within two seconds of the photo. That leaves time to negotiate, or to wave the rickshaw on and wait for the next one, before the driver pulls away.

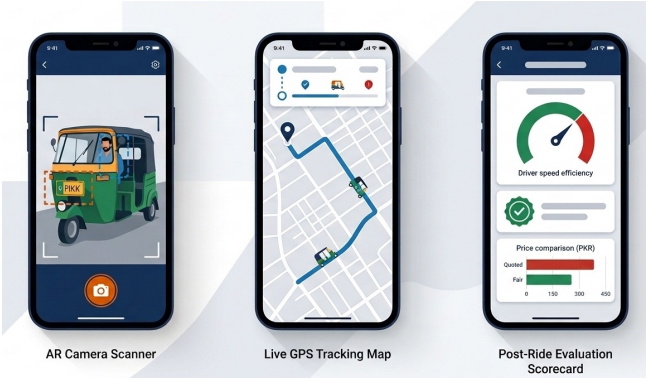


Figure 13: Proposed app flow: AR scan at the curb, live GPS ride tracking, and the post-scan scorecard with pace gauge and quoted-versus-fair price bars.

Deployment also adds safeguards that a classroom model does not need. Drivers will be able to view and dispute their own profile, and a badge appears only after five verified rides. Profiles will weight recent rides more heavily, so a driver who improves is not held back by old data. Because fuel prices change, the fare model will be retrained every month against fresh ride-hailing benchmarks. Every new ride flows back into the Collect stage, which closes the loop shown in Figure 2. Figure 14 shows the eight-week plan from prototype to campus beta.

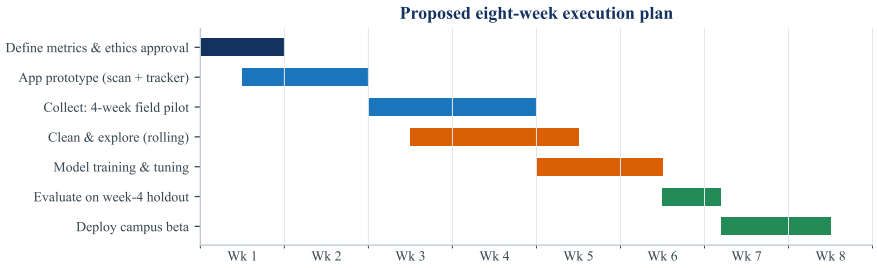


Figure 14: Proposed execution timeline. The four-week field pilot supplies the data for every later stage.

Conclusion

Our rickshaw story contained a problem for every stage of the lifecycle. The simulated pilot suggests that RahiScan can identify drivers reliably by combining plate and face, and that it can price fares to within about 11 rupees. It also suggests that drivers can be judged fairly on pace only when speed is compared with traffic and averaged over several rides. With real rides in place of mock ones, the same pipeline in our repository would give Lahore's student commuters the information they currently lack at the curb.